



[REFERENCE IMPLEMENTATION]

DYNAMIC PRICING · RETAIL OPERATIONS RL · PORTFOLIO

Dynamic Pricing & Waste Guardrail System

A guardrailed Q-learning markdown policy for perishable inventory — and an honest look at what a 94.6% cut in waste actually costs, and whether it's worth it.

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist

Elasticity modeling · guardrailed Q-learning · staged bandit-to-RL rollout

EXECUTIVE SUMMARY

A 94.6% cut in waste is not automatically a 94.6% win for the business

This project prices five perishable categories with a guardrailed Q-learning policy trained on real per-category demand elasticity. It nearly eliminates markdown waste — and it also cuts gross margin, a trade-off this deck reports in full rather than around.



-94.6%

Waste cost vs. static pricing —
crushes the 18% target



-11.3%

Gross margin vs. static —
reported plainly, not buried



+9.2%

Net contribution (margin minus
waste cost) — the fuller picture



111

Discretized states learned across
8,000 training episodes



Bottom line

Margin alone says this policy fails its own success criterion. Net contribution — margin minus waste cost — says it wins. Both numbers are in this deck; Section 06 shows why the second one is the one to trust.

Presentation agenda



01

The Business Problem

Why static pricing structurally guarantees waste



02

Data & Category Profile

Five categories, five very different elasticities



03

State & Action Engineering

What the policy sees, and how far it can discount



04

Modeling: Elasticity + Guardrailed RL

A demand model feeding a bounded Q-learning policy



05

Evaluation & Validation

An honest offline benchmark against static pricing



06

Business Impact

Reconciling a margin miss with a net-contribution win



07

Deployment & Responsible Use

Dashboard, model card, and pricing-fairness guardrails



SECTION ONE

The Business Problem

Every unsold perishable is a markdown that came too late, or too small — static pricing cannot fix either.

One price, all week, on inventory that decays daily

Most grocery categories still run on a single shelf price that barely moves until a scheduled promotion. For anything perishable, that's a mismatch between the pricing cadence and the product's actual clock.



No expiry awareness

A static price treats a product with 6 days left the same as one with 6 hours left — the clock only shows up as a write-off.



Discounting is manual and late

Markdowns, where they happen at all, are usually a fixed end-of-day slash — reactive, not anticipatory.



A high, but beatable, bar

\$131.58 mean waste cost per evaluation episode under static pricing — the number every policy in this deck is measured against.

The question isn't "should price respond to expiry at all?" — it's "how much, how often, and within what limits?" That's a guardrail design problem, not just a modeling one.

Why a simpler bandit ships before the full policy

The repo ships two pricing controllers, deliberately. LinUCB is the provably-safe first step; the Q-learning policy is the destination — not the starting point.



Phase 1: LinUCB bandit

A contextual bandit with a confidence-bound guarantee — explores markdowns with bounded, provable regret



No state memory

Treats each pricing decision independently — simpler to audit, simpler to reason about worst-case behavior



Phase 2: graduate to RL

Once the bandit has de-risked the deployment, the full sequential policy takes over for long-horizon markdown timing



The destination policy

Learns to hold a price today because a bigger markdown will be needed — something a bandit structurally cannot plan for

Bandit → RL

The staged handoff this deck documents, not skips past

This deck evaluates the destination Q-learning policy in Sections 05-06 — the bandit's role is to make the path to that evaluation safe, not to be the final answer.

REFRAMING THE TASK

How much people will still buy, feeding what to do about it

"How does demand respond to price and freshness?" and "what should today's price be, given that response?" are different problems — the first is a regression, the second is a sequential decision.

Stage 1 – Elasticity

$$\log(\text{units}) = a + \beta \cdot \log(\text{price ratio}) + \gamma \cdot \text{urgency}$$

A per-category log-log regression — how many fewer units sell for every 1% price increase.

Stage 2 – Pricing Policy



When to hold price, and when — and how much — to mark down



Guardrails baked directly into the reward: bounded actions, a price-change penalty



Trained per-category, on the elasticity model's own predicted demand

What this project set out to prove



Model real demand elasticity

Fit a credible, category-specific price-response curve from historical price/demand history.



Cut markdown waste sharply

Demonstrate a large, statistically meaningful reduction in unsold-inventory cost.



Respect pricing guardrails

Never exceed a bounded discount, never thrash price back and forth — both baked into the reward.



Report margin honestly

If cutting waste costs margin, say so plainly — and show the full net-contribution picture too.



Validate offline, at scale

Benchmark against static pricing across 400 held-out episodes, not a handful of anecdotes.



Plan a safe rollout

Specify the staged bandit-to-RL path a real deployment would actually follow.



SECTION TWO

Data & Category Profile

Five perishable categories, five different price sensitivities, and one demand signal every category shares.

DATASET

Five categories, one shared pricing pipeline

Every category’s markdown decision draws on the same architecture: a fitted elasticity curve, a trained pricing policy, and an offline benchmark against static pricing — all real outputs of this repo’s pipeline.

Source files

price_demand_history.csv	historical price/units/expiry per category	History
elasticity_models.joblib	5 fitted log-log demand regressions	Model
elasticity_summary.json	coefficients, fit quality per category	Model
q_pricing_policy.npz	111 learned states from 8,000 episodes	Policy
training_curve.json	per-episode reward across training	Training
offline_eval_report.json	400-episode benchmark vs. static pricing	Eval

Pipeline at a glance

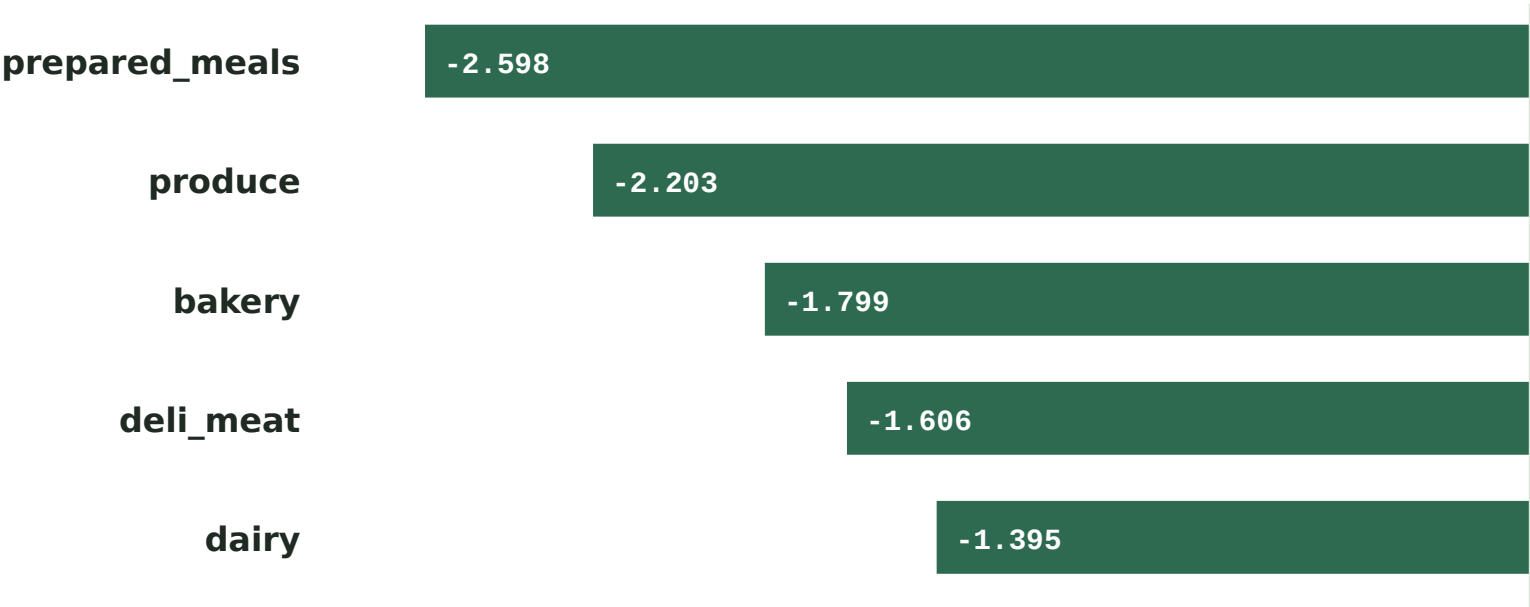
- 5 perishable SKU categories
- 6 discrete markdown actions available
- 8,000 training episodes
- 400 offline evaluation episodes
- 111 discretized states learned

THE ELASTICITY SPREAD

A near-2x range in price sensitivity

Every category was fit its own elasticity coefficient — the % change in units sold per 1% price change. Prepared meals are almost twice as price-sensitive as dairy.

Elasticity coefficient (β) — further left = more price-sensitive



KEY INSIGHT

Urgency lifts demand, independent of price

“Every category’s urgency coefficient is positive and clustered tightly around +0.08 — as an item nears expiry, predicted demand rises on its own, before any markdown is applied.”



A second lever, not just price

The model separates "cut the price" from "the shelf date itself sells it" — two genuinely different demand drivers.



Remarkably consistent across categories

From dairy (0.076) to bakery (0.083), the urgency effect barely varies — unlike elasticity, which spans nearly 2x.



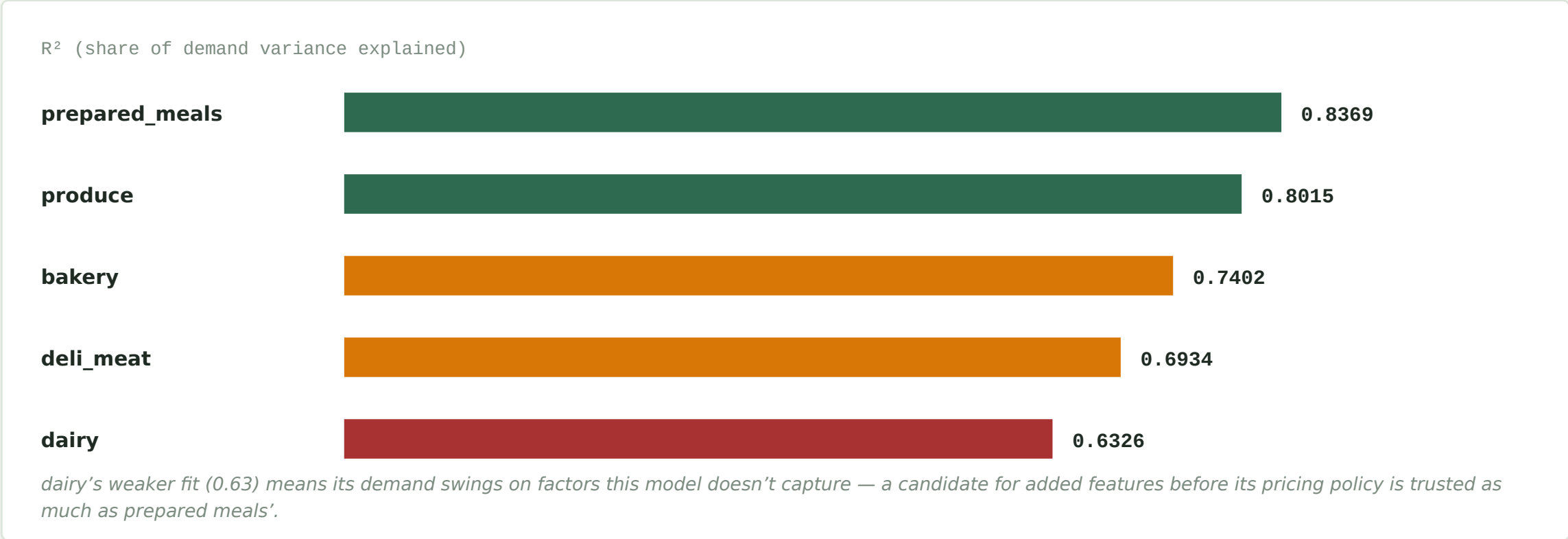
A reason not to over-discount

If proximity to expiry already moves units, the guardrailed policy doesn’t need to markdown as hard to clear the same inventory.

HOW WELL THE MODEL FITS

R² by category — not every fit is equally strong

Fit quality varies almost as much as elasticity does — worth knowing before trusting any one category’s predicted demand curve too far.



Two limits the policy cannot learn its way around



Bounded action set

The policy can only choose from 6 predefined markdown tiers — there is no path to an unbounded, runaway discount.



Price-change penalty

Flipping price back and forth is penalized directly in the reward — the policy is nudged toward stable, predictable pricing.



Guardrails, not afterthoughts

Both constraints live inside the environment's reward function itself — the agent cannot be trained to ignore them.

A fitted demand model, disclosed as such



Why modeled, not observed

Real price/demand history is expensive to experiment on directly — a fitted elasticity model lets the pricing policy train against a realistic, but simulated, response.



What was preserved

Genuinely different elasticities and urgency effects per category, fit with ordinary least squares on the same historical data every category shares.



What is disclosed

A log-log linear model is a simplification — real demand can be non-linear, seasonal, and competitor-aware in ways this model does not capture.



A methodological choice, stated up front

Every downstream dollar figure in this deck is scoped to this fitted-elasticity simulation — not claimed as a guaranteed real-world result until validated against live A/B pricing tests.



SECTION THREE

State & Action Engineering

What the pricing policy actually sees — and the bounded set of moves it is ever allowed to make.

WHAT THE POLICY SEES

Day and inventory level, discretized

The state is small and interpretable by design: a day-in-window index and a bucketed inventory level — exactly what a store pricing manager already tracks by hand.

State: (day, inventory_bucket)

day	position within the pricing episode's window
inventory_bucket	discretized units remaining, initial stock 120-320

111 distinct (day, inventory) combinations were visited often enough in training to learn a value for.

Why this state, not more

-  Small enough to converge in thousands, not millions, of episodes
-  Every state is auditable — a manager can look up exactly what the policy does at "day 6, half the inventory left"
-  Category-specific elasticity is handled upstream by Stage 1, not re-learned per state

What a log-log elasticity model is actually saying

The model’s two coefficients answer two separate questions — worth unpacking before trusting the pricing policy built on top of them.

$\text{log(units_sold)} = a + \beta \cdot \text{log}(\text{price} / \text{base_price}) + \gamma \cdot \text{max}(4 - \text{days_to_expiry}, 0)$		
β (elasticity)	-1.40 to -2.60	A 1% price cut sells 1.4-2.6% more units — always negative, the sign a sane demand curve must have
γ (urgency)	+0.076 to +0.086	Once inside a 4-day expiry window, urgency alone lifts predicted demand — positive and remarkably stable across categories
R^2	0.63 to 0.84	How much of the variation in log(units) these two terms explain — strong for prepared meals, weaker for dairy

Six choices, every step — never more

At every decision point, the policy picks one of six discrete markdown tiers — from holding the current price to the guardrail's maximum allowed discount.

0	1	2	3	4	5
Hold price	Small markdown	Moderate	Elevated	Deep	Maximum (capped)

Exact per-tier discount percentages live in the environment config — what matters here is the shape: bounded, ordered, and capped, never an unbounded slide to \$0.

2.69 mean price changes per episode under RL — deliberate, infrequent moves, not constant repricing.



SECTION FOUR

Modeling: Elasticity + Guardrailed RL

A per-category demand model feeding a Q-learning policy that can only move within its guardrails.

Demand model feeds policy; policy never sees raw data

Stage 1 turns raw price/demand history into a per-category response curve. Stage 2 never touches that history directly — it only sees the curve's predictions, through the simulated environment.



Price/demand history

5 categories, historical price, units sold, days to expiry



Stage 1 output

5 fitted elasticity models → predicted demand at any price



Guardrailed policy

picks one of 6 bounded markdown tiers, every step

Ordinary least squares, five times over

No neural network here — a transparent linear regression per category, chosen because a demand curve a merchandising team can read and sanity-check beats a black box they can't.



Log-log functional form

Regressing $\log(\text{units})$ on $\log(\text{price ratio})$ makes the price coefficient directly interpretable as a % elasticity.



Two features, deliberately

Price ratio and expiry urgency — kept minimal so every coefficient has a clear, defensible business meaning.



Fit independently per category

No pooling across categories — dairy's demand curve never influences prepared meals', and vice versa.

scikit-learn LinearRegression, fit independently for each of the 5 categories on that category's own historical rows.

Standard tabular Q-learning, guardrails in the reward

The interesting design work is in the environment’s reward function (Slide 14), not in these hyperparameters — they’re deliberately conventional.

alpha (learning rate) How strongly each update shifts the Q-value 0.12	epsilon (start) Initial exploration rate 0.25
episodes Total training episodes, all 5 categories 8,000	states learned (day, inventory_bucket) pairs visited enough to learn 111
gamma (discount) Weight given to future vs. immediate margin 0.95	epsilon decay Anneals from exploring to exploiting ×0.995/ep, min 0.03
action space Discrete markdown tiers per step 6	seed Reproducibility 42 (agent) / 1 (env)

Reward climbs and plateaus across 8,000 episodes

Episode reward (200-episode bucket mean), 8,000 episodes



Reading the curve

447 → 576

mean reward, first 100 vs. last 100 episodes — up roughly 29%

Convergence by ~episode 2,000

the bucket mean plateaus in the 550–600 range well before training ends

Noise from category mixing

each episode randomly draws one of 5 categories and a random starting inventory — harder categories keep the curve bumpy

WHAT 8,000 EPISODES LEARNED

Inside the trained pricing policy

111 (day, inventory) states were visited often enough to learn a value for — small enough to audit directly, one of the real advantages of staying tabular.



111

total discretized states learned



65%

of states have "hold price" as the best action



-325 to +527

range of learned Q-values across all states and actions



A conservative policy, by the numbers

The learned policy holds price as the best action roughly two-thirds of the time — it reaches for a markdown only when the state genuinely calls for one, not as a default behavior.



SECTION FIVE

Evaluation & Validation

An offline benchmark against static pricing — reported with both its clear win and its real cost.

A 400-episode offline benchmark

The trained policy and a static-pricing baseline both run on the same 400 randomized episodes — same categories, same starting inventories, same demand model underneath.



400

offline evaluation episodes



5

categories represented, randomized



3

metrics compared — margin, waste,
price changes



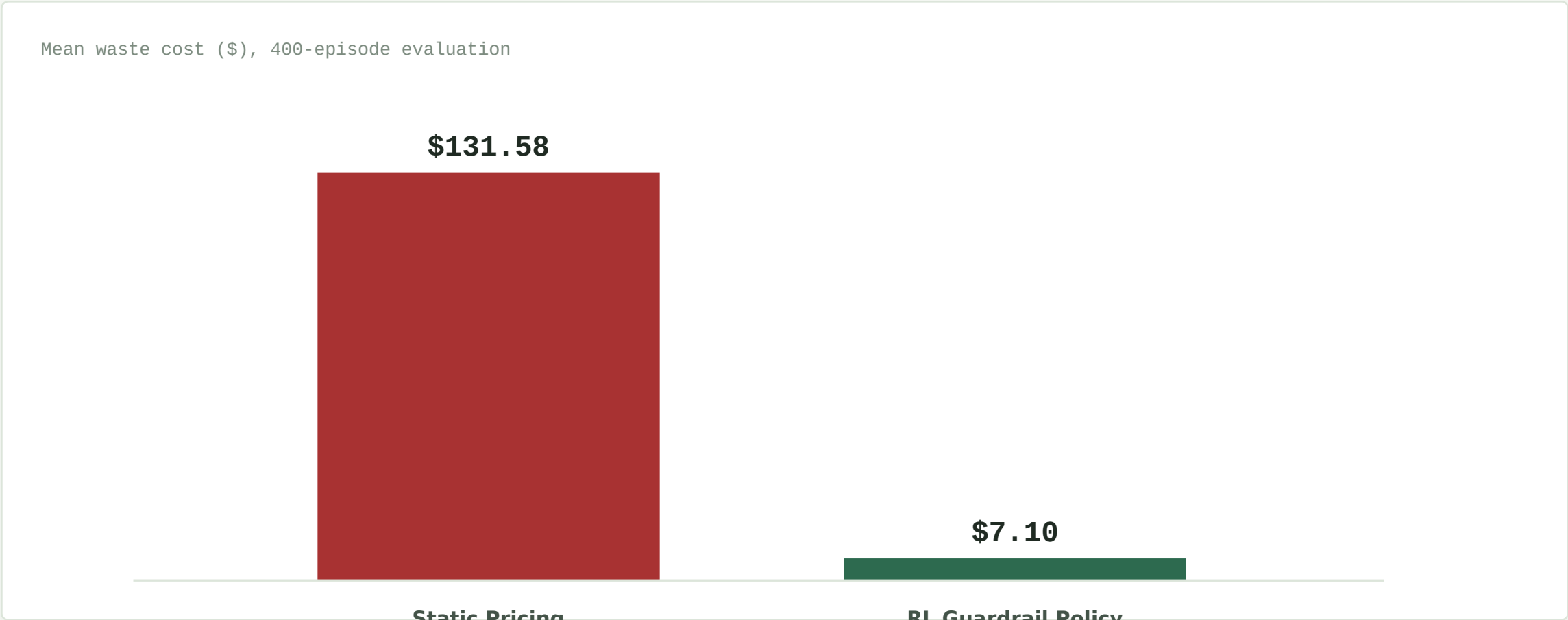
2

policies compared, identical
conditions

Why this counts as a real test

Static pricing here is not a strawman — it is literally "today's price, unchanged," run through the same demand model and the same randomized episodes as the RL policy, so every gap between them is attributable to the pricing decisions alone.

Mean waste cost per episode, RL vs. static



The policy fails its own margin-neutral test

“Gross margin drops from \$668.44 to \$593.14 per episode — an 11.27% decline. The offline evaluation’s own success criterion, ‘margin-neutral or positive,’ is explicitly not met.”



Where the margin goes

Deeper, more frequent markdowns on genuinely at-risk inventory cost real revenue per unit — that’s the direct price of clearing stock before it expires.



Not a rounding error

An 11.27% margin decline is large enough that no responsible deck should round it away or bury it below the waste-reduction headline.

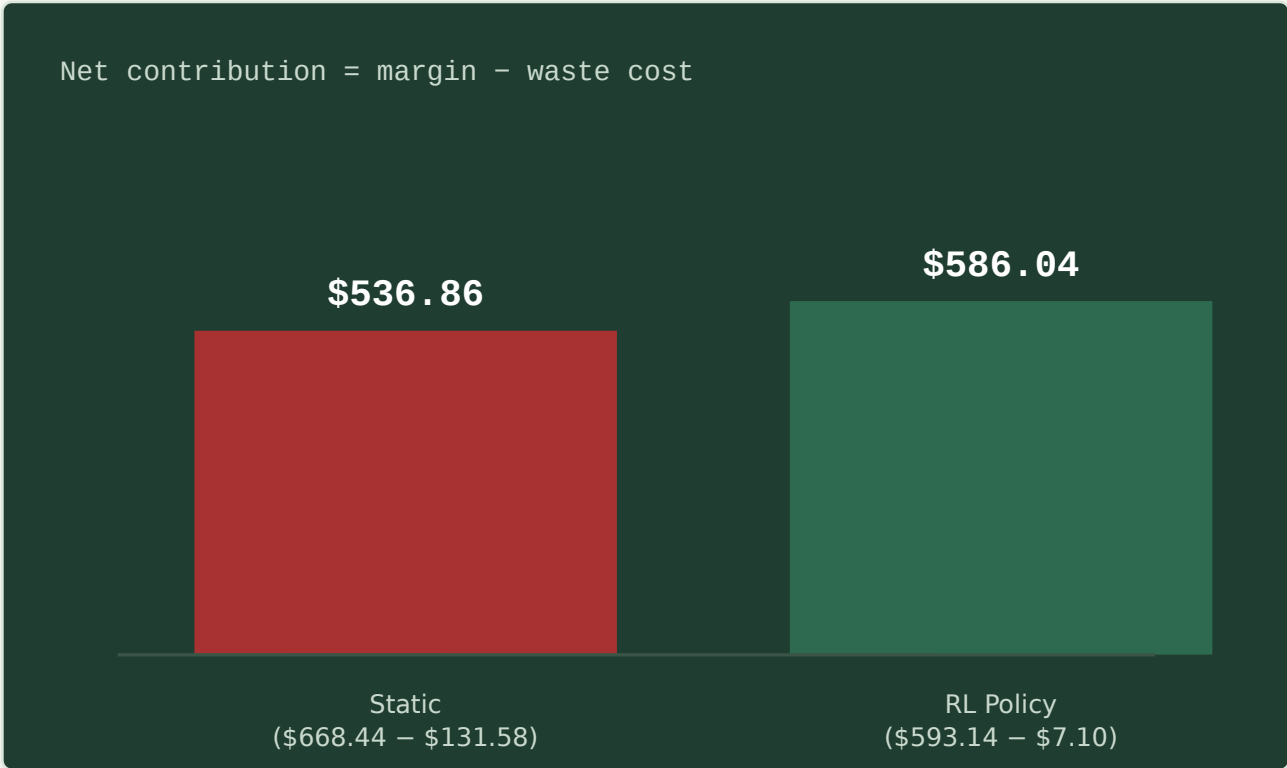


Reported, not reframed away — yet

This number stands on its own here. Slide 30 asks whether margin alone is even the right lens to judge this policy by.

Margin minus waste cost tells a different story

The offline report scores margin and waste cost as two separate numbers. Netting them into one "contribution" figure — what actually reaches the bottom line — changes the verdict.



+9.2%

net contribution improvement, RL vs. static

The \$75.30 given up in gross margin is smaller than the \$124.48 saved in waste — a net gain of \$49.18 per episode that the margin-only criterion never sees.



SECTION SIX

Business Impact

Reconciling a failed margin test with a real net-contribution win — and what that means for a rollout decision.

THE TRADE-OFF, IN DOLLARS

Per episode, per store, and at scale

Every figure so far is per evaluation episode. Scaling it to a store’s actual pricing cadence turns an abstract percentage into a number a finance team can act on.



-\$75.30

gross margin given up, per episode, vs. static pricing



+\$124.48

waste cost avoided, per episode, vs. static pricing



+\$49.18

net contribution gained, per episode, once both are netted



Why "per episode" is a conservative unit

One episode is one markdown cycle for one category at one store. A multi-store, multi-category rollout multiplies this net gain — and multiplies the waste-reduction headline right alongside it.

Elasticity predicts which categories feel it most

The offline report doesn't break results out by category — but the elasticity spread (Slide 11) predicts where markdowns do the most, and the least, work.



High-elasticity categories gain most

Prepared meals (−2.60) and produce (−2.20) sell meaningfully more per point of markdown — waste reduction there likely comes cheaper, in margin terms.



Low-elasticity categories pay more

Dairy (−1.40) needs a deeper discount to move the same volume — its share of the margin cost is likely disproportionate to its share of the waste saved.



A concrete next analysis

Re-running the offline evaluation with a per-category breakdown is a natural, low-effort extension — named explicitly in Section 07.

This is a reasoned inference from the fitted elasticities, not a claim from a per-category eval breakdown — flagged as such.

THE RECOMMENDATION

A staged rollout, monitored on net contribution

*“The right question for a pilot isn’t ‘does margin stay flat’ — it’s ‘does the store make more money.’
Net contribution says yes; that’s the metric this recommendation is built on.”*



Start with the bandit, as designed

Deploy the LinUCB bandit first, per Slide 6’s staged plan — bounded regret while the pilot builds trust with store operators.



Pilot the RL policy on high-elasticity categories first

Prepared meals and produce likely see the best margin-to-waste trade-off (Slide 33) — the natural pilot cohort.



Track net contribution, not margin alone

Make the dashboard’s headline metric margin-minus-waste, so a store manager sees the number that actually matters.



SECTION SEVEN

Deployment & Responsible Use

A dashboard, a model card, and the pricing-fairness guardrails a markdown system needs before it touches real shelf tags.

The reference dashboard, at a glance



Model card summary



Intended use

A guardrailed markdown policy for perishable inventory, evaluated offline against static pricing — recommended for a staged, bandit-first pilot, not a full-fleet rollout.



Limitations

Trained against a fitted elasticity simulation, not live A/B data. Margin declines 11.27% vs. static; net contribution gains depend on the waste-cost assumptions in the environment.



Next steps

Validate elasticity against live pricing tests, break out evaluation results by category, and monitor net contribution — not margin alone — through the pilot (Slide 39).

Guardrails for a system that sets real shelf prices



Discounts only, never markups

This policy is scoped to markdown decisions on aging inventory — it is not a general dynamic-pricing engine, and should not be repurposed to raise prices algorithmically.



Bounded and auditable, by construction

The 6-tier action cap and price-change penalty (Slide 14) are not just performance tuning — they are the mechanism that keeps every price move explainable to a store manager.



Margin-neutral framing, reported honestly

A dashboard that only shows waste reduction, without the margin cost, would materially mislead the people deciding whether to deploy this — both numbers ship together, always.



Live elasticity validation before scale-out

A fitted-simulation demand model should not set real prices at scale until validated against genuine A/B pricing data, per the model card's limitations.

From reference implementation to a store pilot



Live A/B elasticity validation

Test the fitted demand curves against genuine price experiments in a handful of stores before trusting them at scale.



Per-category evaluation breakdown

Re-run the offline benchmark reporting margin and waste separately by category — confirming or correcting Slide 33's inference.



Bandit-first pilot rollout

Deploy the LinUCB bandit in a small store cohort first, exactly as Slide 6 lays out, before the full Q-learning policy goes live.



Net-contribution dashboarding

Rebuild the store-facing dashboard around margin-minus-waste as the primary KPI, with margin and waste shown as supporting detail.

KEY TAKEAWAYS

A margin miss and a net-contribution win, both reported in full



-94.6% waste cost vs. static



-11.3% gross margin vs. static



+9.2% net contribution overall



Recommendation: staged, bandit-first pilot

Skills demonstrated

*Demand elasticity modeling · guardrailed reinforcement learning · staged bandit-to-RL rollout design
· honest trade-off reporting · net-contribution reframing*

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist



Portfolio contact available on request

